

# Chapitre 8

## Logique propositionnelle

### Sommaire.

|     |                               |   |
|-----|-------------------------------|---|
| 1   | Formules propositionnelles.   | 1 |
| 1.1 | Syntaxe.                      | 1 |
| 1.2 | Sémantique.                   | 2 |
| 1.3 | Satisfiabilité.               | 2 |
| 2   | Formes normales.              | 3 |
| 2.1 | Formes normales négatives.    | 3 |
| 2.2 | Formes normales conjonctives. | 3 |
| 2.3 | Algorithme de Quine           | 4 |
| 2.4 | Forme normale disjonctive.    | 5 |
| 3   | Logique des prédicats.        | 6 |

Les propositions marquées de ★ sont au programme de colles.

### 1 Formules propositionnelles.

#### 1.1 Syntaxe.

Définition 1: Logique propositionnelle.

Soit  $\mathbb{V}$  un ensemble fini (ou dénombrable) de symboles appelés variables propositionnelles.

On définit inductivement l'ensemble des formules propositionnelles sur  $\mathbb{V}$  :

- $\perp$  et  $\top$  sont des expressions logiques, **Faux** et **Vrai** respectivement.
- $p$  est une variable propositionnelle de  $\mathbb{V}$ .
- À partir de  $\varphi$  et  $\psi$  deux formules, on peut construire :
  - $\hookrightarrow (\varphi \wedge \psi)$  (conjonction).
  - $\hookrightarrow (\varphi \vee \psi)$  (disjonction).
  - $\hookrightarrow (\neg \varphi)$  (négation).

Dans ce polycopié,  $\varphi$  et  $\psi$  désigneront toujours des formules.

Toute formule propositionnelle peut être représentée par un arbre : avec les variables propositionnelles en tant que feuilles, et les constructeurs en tant que noeuds internes.

## 1.2 Sémantique.

### Définition 2: Valuation.

Une **valuation** sur  $\mathbb{V}$  est une application  $v : \mathbb{V} \rightarrow \{0, 1\}$ , on l'étend aux formules propositionnelles : Soient  $\varphi, \psi$  des formules propositionnelles. On définit inductivement  $v(\varphi)$  tel que :

- $\hookrightarrow v(\perp) = 0$  et  $v(\top) = 1$ .
- $\hookrightarrow v(\varphi) = v(\varphi)$  si  $\varphi \in \mathbb{V}$ .
- $\hookrightarrow v(\varphi \wedge \psi) = v(\varphi) \times v(\psi)$ .
- $\hookrightarrow v(\varphi \vee \psi) = v(\varphi) + v(\psi) - v(\varphi) \times v(\psi)$ .
- $\hookrightarrow v(\neg\varphi) = 1 - v(\varphi)$ .

Dans ce polycopié,  $v$  désignera toujours une valuation.

### Définition 3: Équivalence logique. ★

Deux formules  $\varphi$  et  $\psi$  sont **sémantiquement équivalentes** si pour toute valuation  $v$  sur  $\mathbb{V}$ ,  $v(\varphi) = v(\psi)$ . On note alors  $\varphi \equiv \psi$ . Ainsi,  $\equiv$  est une relation d'équivalence sur les formules.

**Remarque:** Dans la pratique, on compare les tables de vérité de  $\varphi$  et  $\psi$ .

### Définition 4: Autres constructeurs.

Il existe des liens logiques qui s'expriment à partir de ceux de base :

- $\hookrightarrow$  L'implication  $\varphi \rightarrow \psi \equiv (\neg\varphi) \vee \psi$ .
- $\hookrightarrow$  L'équivalence  $\varphi \leftrightarrow \psi \equiv (\varphi \rightarrow \psi) \wedge (\psi \rightarrow \varphi)$ .
- $\hookrightarrow$  Vrai :  $\top \equiv \varphi \vee \neg\varphi$ .
- $\hookrightarrow$  Faux :  $\perp \equiv \varphi \wedge \neg\varphi$ .

### Proposition 5: Lois de De Morgan. ★

Soient  $\varphi$  et  $\psi$  deux formules logiques. Alors :

- $\hookrightarrow \neg(\varphi \wedge \psi) \equiv (\neg\varphi) \vee (\neg\psi)$ .
- $\hookrightarrow \neg(\varphi \vee \psi) \equiv (\neg\varphi) \wedge (\neg\psi)$ .

**Preuve :**

On le montre facilement en comparant les tables de vérités.

## 1.3 Satisfiabilité.

### Définition 6: Modèles, satisfiabilité.

Soit  $\varphi$  une formule sur  $\mathbb{V}$ . Une valuation  $v : \mathbb{V} \rightarrow \mathcal{B}$  est un **modèle** de  $\varphi$  si  $v(\varphi) = 1$ , on note alors  $v \models \varphi$ . On dit alors qu'une formule est **satisfiable** si elle admet un modèle.

Une formule  $\varphi$  pour laquelle toute valuation est un modèle est une tautologie, on note  $\models \varphi$ .

Si aucune valuation n'en est un modèle,  $\varphi$  est une antilogie (pas anthologie), on note  $\not\models \varphi$ .

### Définition 7: Conséquence logique.

Soient  $\varphi, \psi$  deux formules.

On dit que  $\varphi$  est en **conséquence logique** de  $\psi$ , et on note  $\psi \models \varphi$  si tout modèle de  $\psi$  est modèle de  $\varphi$ .

On étend cette notation à un ensemble  $\Gamma$  de formules, dans ce cas, on dit que  $\varphi$  est une **conséquence logique** de  $\Gamma$  si  $\varphi$  est en **conséquence logique** de toute formule de  $\Gamma$ .

**Définition 8: Équisatisfiabilité**

Deux formules  $\varphi$  et  $\psi$  sont **équisatisfiables** si  $\varphi$  est satisfiable si et seulement si  $\psi$  l'est.

**2 Formes normales.****2.1 Formes normales négatives.****Définition 9: Littéral.**

On appelle **littéral** une variable propositionnelle ou sa négation.

**Définition 10: Construction. ★**

Une formule est dite en **forme normale négative (FNN)** si ses négations ne s'appliquent qu'aux variables. Pour une formule  $\varphi$ , on construit sa forme normale négative  $\text{nnF}(\varphi)$  inductivement de la manière suivante :

- $\hookrightarrow \text{nnF}(\varphi) = \varphi$  si c'est un littéral.
- $\hookrightarrow \text{nnF}(\neg\neg\varphi) = \text{nnF}(\varphi)$
- $\hookrightarrow \text{nnF}(\varphi \wedge \psi) = \text{nnF}(\varphi) \wedge \text{nnF}(\psi)$
- $\hookrightarrow \text{nnF}(\varphi \vee \psi) = \text{nnF}(\varphi) \vee \text{nnF}(\psi)$
- $\hookrightarrow \text{nnF}(\neg(\varphi \vee \psi)) = \text{nnF}(\neg\varphi) \wedge \text{nnF}(\neg\psi)$
- $\hookrightarrow \text{nnF}(\neg(\varphi \wedge \psi)) = \text{nnF}(\neg\varphi) \vee \text{nnF}(\neg\psi)$

**Proposition 11: Existence. ★**

Pour toute formule  $\varphi$ ,  $\text{nnF}(\varphi)$  est sous forme normale négative et  $\text{nnF}(\varphi) \equiv \varphi$ .

**Preuve :**

Par induction sur les formules propositionnelles.

**Cas de base.** Soit  $\varphi$  un littéral.  $\text{nnF}(\varphi) = \varphi$  sous FNN et  $\text{nnF}(\varphi) \equiv \varphi$ .

**Hérédité:** Soient  $\varphi, \psi$  telles que la propriété soit vraie sur elles-mêmes et leurs négations.

Soit  $v$  une valuation de  $\varphi$  et  $\psi$ .

On a  $\text{nnF}(\varphi \wedge \psi) = \text{nnF}(\varphi) \wedge \text{nnF}(\psi)$  donc c'est bien sous forme normale négative par hypothèse.

De plus,  $v \models \text{nnF}(\varphi \wedge \psi) \iff v \models \text{nnF}(\varphi) \wedge \text{nnF}(\psi) \iff v \models \varphi \text{ et } v \models \psi \iff v \models \varphi \wedge \psi$ .

On a  $\text{nnF}(\neg(\varphi \wedge \psi)) = \text{nnF}(\neg\varphi) \vee \text{nnF}(\neg\psi)$  donc c'est bien sous forme normale négative par hypothèse.

De plus,  $v \models \text{nnF}(\neg(\varphi \wedge \psi)) \iff v \models \text{nnF}(\neg\varphi) \vee \text{nnF}(\neg\psi) \iff v \models \neg\varphi \text{ ou } v \models \neg\psi \iff v \models \neg\varphi \vee \neg\psi \iff v \models \neg(\varphi \wedge \psi)$ .

Même raisonnement pour la disjonction.

Par théorème d'induction, c'est vrai pour toute formule  $\varphi$ .

**2.2 Formes normales conjonctives.****Définition 12: Problème SAT.**

Le problème SAT prend une formule en entrée et répond à la question : "Cette formule est-elle satisfiable ?".

**Définition 13: Clause.**

Une **clause** est une disjonction de littéraux.

**Définition 14: Forme normale conjonctive. ★**

Une formule est en **forme normale conjonctive (FNC)** si elle est une conjonction de clauses.

On définit inductivement la mise sous FNC de  $\varphi$  en  $\text{cnF}(\varphi)$  par :

- $\hookrightarrow \text{cnF}(\varphi) = \varphi$  si  $\varphi$  littéral.
- $\hookrightarrow \text{cnF}(\varphi \vee \psi) = \varphi \vee \psi$  si  $\varphi, \psi$  littéraux.
- $\hookrightarrow \text{cnF}(\varphi \wedge \psi) = \text{cnF}(\varphi) \wedge \text{cnF}(\psi)$ .
- $\hookrightarrow \text{cnF}(\varphi \vee (\psi \wedge \psi')) = \text{cnF}(\varphi \vee \psi) \wedge \text{cnF}(\varphi \wedge \psi')$ .
- $\hookrightarrow \text{cnF}(\varphi \vee (\psi \vee \psi')) = \text{cnF}(\varphi \vee \text{cnF}(\psi \vee \psi'))$ .

**Proposition 15**

Si  $\varphi$  est une formule sous FNN,  $\text{cnF}(\varphi)$  est sous FNC et  $\text{cnF}(\varphi) \equiv \varphi$ .

**Preuve :**

Même principe de preuve que pour la FNN.

**Proposition 16**

Si  $\varphi$  est sous FNN, on peut construire une FNC équisatisfiable à  $\varphi$  en temps linéaire.

**2.3 Algorithme de Quine****Définition 17: Substitution.**

Soit  $\varphi$  une formule sur un ensemble  $\{p_1, \dots, p_n\}$  et soient  $\{\varphi_1, \dots, \varphi_n\}$  des formules.

La substitution des  $\varphi_i$  aux  $p_i$  est la formule obtenue en remplaçant simultanément chaque  $p_i$  par  $\varphi_i$ .

On la note  $\varphi[\varphi_1/p_1, \dots, \varphi_n/p_n]$ .

La substitution se définit inductivement :

- $\varphi[\varphi_i/p_i] = \varphi_i$  si  $\varphi = p_i$ .
- $\varphi[\varphi_1/p_1, \dots, \varphi_n/p_n] = \neg\varphi'[\varphi_1/p_1, \dots, \varphi_n/p_n]$  si  $\varphi = \neg\varphi'$ .
- $\varphi[\varphi_1/p_1, \dots, \varphi_n/p_n] = \varphi_1[\varphi_1/p_1, \dots, \varphi_n/p_n] \wedge \varphi_2[\varphi_1/p_1, \dots, \varphi_n/p_n]$  si  $\varphi = \varphi_1 \wedge \varphi_2$ .
- $\varphi[\varphi_1/p_1, \dots, \varphi_n/p_n] = \varphi_1[\varphi_1/p_1, \dots, \varphi_n/p_n] \vee \varphi_2[\varphi_1/p_1, \dots, \varphi_n/p_n]$  si  $\varphi = \varphi_1 \vee \varphi_2$ .

**Proposition 18**

Une substitution dans une tautologie donne une tautologie.

**Preuve :**

Soit  $\varphi$  sur  $\{p_1, \dots, p_n\}$ ,  $\{\varphi_1, \dots, \varphi_n\}$  des formules sur  $\mathbb{V}$ ,  $v$  une valuation sur  $\mathbb{V}$  et  $\omega$  sur  $\{p_1, \dots, p_n\} : \omega(p_i) = v(\varphi_i)$ . Montrons que  $\omega(\varphi) = v(\varphi[\dots])$ . (on écrira ... au lieu de  $\varphi_1/p_1, \dots, \varphi_n/p_n$  à partir de maintenant)

**Cas de base.** Trivial si  $\varphi = \top$  ou  $\varphi = \perp$ . Si  $\varphi = p_i$ , alors  $\varphi[\dots] = \varphi_i$  et  $\omega(\varphi) = \omega(p_i) = v(\varphi_i)$ .

**Hérédité.** Si  $\varphi = \neg\varphi'$ ,  $\omega(\varphi) = \omega(\neg\varphi') = \neg\omega(\varphi') = \neg v(\varphi'[\dots]) = v(\neg\varphi'[\dots]) = v(\varphi[\dots])$ .

Si  $\varphi = \varphi_1 \vee \varphi_2$ ,  $\omega(\varphi) = \omega(\varphi_1 \vee \varphi_2) = \omega(\varphi_1) \vee \omega(\varphi_2) = v(\varphi_1[\dots]) \vee v(\varphi_2[\dots]) = v(\varphi_1[\dots] \vee \varphi_2[\dots]) = v(\varphi)$ .

De même pour la conjonction, avec  $\varphi_1, \varphi_2$  vérifiant l'hypothèse. Par théorème d'induction structurale, la propriété est vérifiée.

Supposons  $\varphi$  une tautologie. Soit  $v$  une valuation de la formule substituée., il existe  $\omega$  telle que  $\omega(\varphi) = v(\varphi[\dots])$ . Comme  $\varphi$  est tautologie,  $\omega(\varphi) = 1$  donc  $v(\varphi[\dots]) = 1$  donc  $v \models \varphi[\dots]$ , c'est une tautologie.

**Définition 19: Algorithme de Quine.****Entrée:**  $\varphi$  sous FNC.**Sortie:** 1 si  $\varphi$  est satisfiable, 0 sinon.

1. Simplifier les clauses.
2. Si  $\varphi$  est une conjonction sur  $\emptyset$ , renvoyer 1.
3. Si  $\varphi$  contient  $\perp$ , renvoyer 0.
4. Choisir la prochaine variable  $p$  dans l'une des clauses :  
 $\hookrightarrow$  Si Quine( $\varphi[\perp/p]$ ), renvoyer 1, sinon renvoyer Quine( $\varphi[\top/p]$ ).

**Étape 1:**

1. Si la clause est  $\top$ , la supprimer.
2. Tiers-exclu : les clauses contenant des littéraux opposés sont supprimées.
3. Fusion : supprimer les doublons de littéraux.
4. Si une clause en contient une autre, on la supprime.
5. Si une clause contient  $\perp$ , le supprimer.

**Terminaison:** Toutes les opérations s'effectuent en temps fini.

Il y a un nombre fini d'appels récursifs : variant d'appel donnée par le nombre de variables apparaissant dans la formule.

**Correction:** assurée par le tiers-exclu.**2.4 Forme normale disjonctive.****Définition 20: Conjonction élémentaire.**Une **conjonction élémentaire** est une formule sans disjonctions.**Définition 21: Forme normale disjonctive. ★**Une formule est une **forme normale disjonctive (FND)** si c'est une disjonction de conjonctions élémentaires.Pour passer de  $\varphi$  sous FNN à  $\text{dnF}(\varphi)$  sous FND, on procède par induction :

- $\text{dnF}(\varphi) = \varphi$  si  $\varphi$  est littéral.
- $\text{dnF}(\varphi) = \varphi$  si  $\varphi = l \wedge l'$  avec  $l, l'$  littéraux.
- $\text{dnF}(\varphi \vee \psi) = \text{dnF}(\varphi) \vee \text{dnF}(\psi)$ .
- $\text{dnF}(\varphi \wedge (\psi \vee \psi')) = \text{dnF}(\varphi \wedge \psi) \vee \text{dnF}(\varphi \wedge \psi')$ .
- $\text{dnF}(\varphi \wedge (\psi \wedge \psi')) = \text{dnF}(\varphi \wedge \text{dnF}(\psi \wedge \psi'))$ .

**Proposition 22**Si  $\varphi$  est sous FNN,  $\text{dnF}(\varphi)$  est sous FND et  $\text{dnF}(\varphi) \equiv \varphi$ .**Preuve :**Pour tout modèle  $v$  de  $\varphi$ , on construit :

$$\varphi_v = \bigwedge_{p \in \mathbb{V}} l_p \quad \text{où} \quad l_p = \begin{cases} p & \text{si } v(p) = 1 \\ \neg p & \text{sinon} \end{cases}.$$

On pose alors  $\psi$  la disjonction des  $\varphi_v$  pour tout modèle  $v$  de  $\varphi$ , alors  $\psi$  sous FND et  $\psi \equiv \varphi$ .**Définition 23**

Une FND est complète si chaque variable est représentée une unique fois dans chaque conjonction élémentaire.

### 3 Logique des prédicats.

#### Définition 24

Pour chaque instance de logique du premier ordre, on introduit un ensemble de symboles:

- ↪ Une infinité de symboles de variables.
- ↪ Des symboles de constantes, éléments particuliers du domaine d'interprétation.
- ↪ Des symboles de fonctions, transforment les tuples d'éléments.
- ↪ Des symboles de prédicats, expriment des propriétés sur les éléments.

#### Exemple 25

Un groupe peut être décrit par:

- ↪ Constantes:  $e$  le neutre.
- ↪ Fonctions: le produit, l'inverse.
- ↪ Prédicat: l'égalité.

Familles des ensembles:

- ↪ Constantes:  $\emptyset$
- ↪ Fonctions:  $\cup, \cap, \setminus$
- ↪ Prédicats:  $\in, \subset, =$ .

#### Définition 26: Termes.

À partir des fonctions, des constantes et des variables, on définit inductivement les **termes**:

- ↪ Assertions: constantes et variables.
- ↪ Règles d'inférence: si  $f$  est une fonction d'arité  $n$ , et  $t_1, \dots, t_n$  des termes,  $f(t_1, \dots, t_n)$  est un terme.

#### Définition 27: Atomes.

Un **atome** (ou formule atomique) est l'application d'un prédicat  $n$ -aire à une suite de  $n$  termes.

#### Définition 28: Formules.

Les **formules** de la logique des prédicats sont construites inductivement:

- ↪ Assertions: atomes.
- ↪ Règles d'inférence:  $\wedge, \vee, \neg, \exists, \forall$ .

#### Exemple 29: Groupes abéliens.

Dans un groupe abélien, on voudrait que les formules suivantes soient vraies:

$$\forall x. x + 0 = x, \quad \forall x. \exists y. x + y = 0, \quad \forall x. \forall y. x + y = y + x$$

#### Définition 30: Occurences.

Une occurrence d'une variable  $x$  dans une formule  $F$  est une position de cette variable dans  $F$ .

Une occurrence de  $x$  est liée dans  $F$  si dans la branche qui aboutit à cette occurrence on rencontre une quantification de  $x$ . Sinon elle est libre.

#### Exemple 31

Dans  $\exists x. P(x, y)$ ,  $x$  est liée et  $y$  est libre.

#### Définition 32

Deux formules  $F$  et  $G$  sont  $\alpha$ -équivalentes si on peut passer de l'une à l'autre en renommant des variables liées.